

문제 해결 : Plus - Manager

시스템 아키텍처	
프로젝트 배경	
기능 설명	
문제 해결	

기존 구조 및 문제점

```
private static final String PROMPT_TEXT = """"
CRITICAL INSTRUCTION
YOU MUST OUTPUT **ONLY VALID JSON**.\s
DO NOT output plain text, explanations, or anything except JSON.
ANY non-JSON output will cause SYSTEM FAILURE.
(이하 생략)
""";
```

- 서버는 AI 응답이 JSON이라는 것을 전제로 데이터 저장 로직 수행
- LLM이 텍스트+JSON으로 반환하는 경우 서버에 문제가 발생
- JSON 파싱 단계에서 JsonParseException가 발생
- 프롬프트에 의존한 구조로, 응답 형식에 대한 강제력이 없는 설계

문제 해결 방법

```
private final BeanOutputConverter<UnifiedResponse> converter;
//(이하 생략...)
.options(
    OpenAiChatOptions.builder()
        .model(MODEL_ID)
        .temperature(0.1)
        .build()
)
.message(messages)
.call()
.entity(converter);
//(이하 생략...)
```

- BeanOutputConverter를 활용해 DTO 기반 JSON Schema를 생성하고, AI 응답을 구조화된 형식으로 제한
- Spring AI의 entity() 방식을 사용하여 AI 응답을 DTO로 직접 매핑하도록 구현
- AI 응답을 문자열로 분석하던 방식을 제거하고, 정해진 구조의 데이터로 받아 오류 발생 가능성을 줄임

문제 해결 : Plus - Manager

트랜잭션 범위 재설정 : 트랜잭션안에 외부 서버와 통신하는 과정을 제거하였습니다

시스템 아키텍처

프로젝트 배경

기능 설명

문제 해결

기존 구조 및 문제점

트랜잭션 시작 → 게시판 조회 (DB) → 도메인 객체 생성
→ 파일 업로드 (외부 API) → 게시물 저장 (DB)
→ 첨부파일 정보 저장(DB) → 트랜잭션 종료

- 커넥션의 개수가 제한적이라 커넥션을 소유하는 시간이 길어질수록 사용가능한 여유 커넥션의 개수가 줄어듦
- 외부 서버와 통신할 수 없는 상황이 발생한다면 타임아웃까지 대기하며 그동안 커넥션을 계속 점유
- 커넥션 고갈 시 신규 요청들이 대기하며 누적되어 DBMS 부하 증가 및 전체 시스템 응답 지연 발생

문제 해결 방법

게시판 조회 (DB) → 도메인 객체 생성
→ 파일 업로드 (외부 API) → 트랜잭션 시작
→ 게시물 저장 (DB) → 첨부파일 정보 저장(DB)
→ 트랜잭션 종료

- DB 저장로직만 트랜잭션 범위로 재설정하여 외부 서버와의 통신을 트랜잭션 외부에서 처리
- 트래픽 증가 시 발생할 수 있는 커넥션 고갈을 사전에 예방
- DBMS 서버 부하 감소

문제 해결 : Plus - Manager



인덱스 없는 UPDATE : 인덱싱을 통하여 Full Table Scan을 방지하였습니다

시스템 아키텍처

프로젝트 배경

기능 설명

문제 해결

기존 구조 및 문제점

- 개발 단계에서 쿼리 점검 중 UPDATE문의 WHERE절에 인덱스가 적용되지 않은 경우 대상 레코드를 찾기 위해 Full Table Scan 이 발생함을 확인
- 어떤 레코드인지 모르기 때문에 테이블 전 범위를 스캔하게 되며 이 과정이 트랜잭션 동안 유지
- SELECT는 차단되지 않지만, 동시 UPDATE 환경에서는 락 대기 및 쓰기 지연이 발생할 가능성이 있다고 판단

문제 해결 방법

after_service (1r × 12c)												
#	id	select_type	table	partitions	type	possible_keys	key	key_len	ref	rows	filtered	Extra
1	1	SIMPLE	a	(NULL)	const	UKksbnjnosmf715ft62ve0ko93i	UKksbnjnosmf715ft62ve0ko93i	82	const	1	100.0	(NULL)

- UPDATE 대상 레코드를 빠르기 식별할 수 있도록 웹 내부적으로 사용되는 publicId 컬럼에 유니크 조건을 적용해 보조 인덱스로 활용
- publicId 기준으로 UPDATE 쿼리가 수행되도록 하여 Full Table Scan 방지
- UPDATE 작동 시 스캔 범위를 줄이고 불필요한 락 발생 가능성을 줄일 수 있도록 개선

시스템 아키텍처	
프로젝트 배경	
기능 설명	
문제 해결	

기존 구조 및 문제점

→ AI 자연어 분석 및 응답 → 주문 DB에 저장

→ 호환 소모품 실시간 검색

→ 주문 정보와 검색 결과 기사한테 문자 발송

- 여러 소모품이 접수되는 경우 각각 호환되는 소모품을 순차적으로 검색하여 사용자가 오래 기다리게되는 문제
- 사용자 입장에서는 불필요한 대기 발생
- 외부 API가 느리면 전체가 느려지는 증상
- 검정 토너, 마젠타 토너 둘 다 토너 모델명 자체는 동일하지만 각각 검색하는 중복 검색 증상

문제 해결 방법

● 비동기 이벤트 발생

↓ 주문 DB에 저장

↓ 응답 DTO 반환

↓ 호환 소모품 실시간 검색

↓ 주문 정보와 검색 결과 기사한테 문자 발송

순차 처리		병렬 처리	
최소	6628ms	최소	4039ms
최대	23570ms	최대	7888ms
평균	12137.6ms	평균	5213.4ms

시간 단축

6924.2ms

- 비동기 이벤트 분리: 불필요한 실시간 처리 제거로 응답 속도 향상
- 중복 검색 제거: 키워드 그룹화로 API 호출 절감
- 병렬 실행: CompletableFuture로 검색 시간 약 57% 단축

문제 해결 : 분당구 보건소 조직 관리 시스템

기술 스택

기능 설명

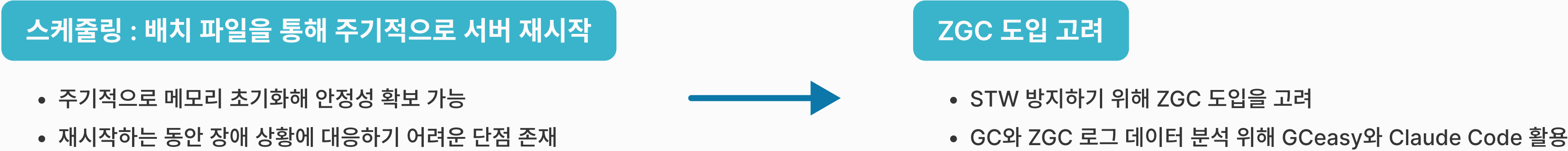
예방 접종

예방 접종 1: 단일 서버 환경을 고려한 GC 튜닝 및 STW(Stop-The-World) 지연 개선

문제점

Garbage Collector가 동작하는 과정에서 Stop The World(STW)로 인해 애플리케이션이 잠시 일시 정지될 수 있음을 인지하였습니다. 폐쇄적인 행정망 내부의 단일 서버 환경에서 해당 지연이 발생할 경우, 전체 서비스 응답 지연으로 이어질 수 있는 가능성을 고려했습니다.

문제 해결



테스트 후 결과 비교

- 1차 테스트 (힙 512MB) : ZGC 적용 직후, 작은 힙 공간으로 인해 객체 생성 속도를 따라가지 못하는 할당 지연 현상이 발생하여 처리량이 89.4%로 저하
- 2차 메모리 튜닝 및 성능 비교 (힙 최대 4GB)
 - : 힙 메모리를 4GB로 확장하여 병목 제거하고 GC와 ZGC의 성능을 동일한 조건에서 테스트
 - : G1 GC → 최대 멈춤 시간 20.0ms / 최대 메모리 사용량 2.59GB
 - : ZGC → 최대 멈춤 시간 0.0390ms / 최대 메모리 사용량 3.91GB

결과

- 단일 서버의 환경 제약을 고려하여 G1 GC 최종 도입
- 16GB 제한된 PC 환경에서 OS, MySQL, Nginx 와 자원 공유해야하므로 ZGC 도입 시 시스템 스와핑 발생 위험
- 메모리 사용을 상대적으로 안정적인 범위로 통제하면서 STW를 20ms 정도로 커버한 G1 GC 를 선택하여 안정성 확보